PROMPTS Y RESULTADOS

Uso de la IA - Prompt 1

Prompt
Selecciono una cantidad y me muestra esa cantidad de colores, el problema viene cuando selecciono por segunda vez una cantida distinta a la primera debido a que no me muestra una nueva cantidad sino la que habia seleccionado primero

Resultado de la IA

El problema es lógico: en tu `generarPaleta()` solo contemplás 2 casos:

1. **No hay colores → crear**
2. **Ya hay colores → actualizar (pero NO cambiar cantidad)**

👉 O sea, nunca agregás ni eliminás divs cuando cambia la cantidad.

🔧 Solución (simple y correcta)

Tenés que comparar la cantidad actual vs la nueva y:

- Si faltan → crear más
- Si sobran → eliminar
- Luego actualizar los no bloqueados

✍️ Reemplazá tu función `generarPaleta` por esta:

JavaScript

```javascript
const actuales = paleta.children.length;

// 🔹 Si faltan colores → agregar
if (actuales < cantidad) {
  for (let i = actuales; i < cantidad; i++) {
    crearColor();
  }
}

// 🔹 Si sobran colores → eliminar desde el final
if (actuales > cantidad) {
  for (let i = actuales; i > cantidad; i--) {
    paleta.removeChild(paleta.lastChild);
  }
}

// 🔹 Actualizar colores no bloqueados
Array.from(paleta.children).forEach((divColor) => {
  if (divColor.dataset.locked !== "true") {
    const nuevoColor = generarColoresAleatorios();
    divColor.style.backgroundColor = nuevoC
```

Uso de la IA - Prompt 2

Prompt
Ahora el problema es que si bloqueo el ultimo de los colores cuando pongo una cantidad menor lo elimina

Resultado de la IA

Ah, perfecto, ahí entendí el detalle. 😅

El problema es que cuando reducís la cantidad, **tu código elimina los divs desde el final sin importar si están bloqueados**. Entonces si bloqueaste el último, igual lo borrás.

La solución es **eliminar solo los divs desbloqueados** y de manera ordenada (del final hacia el inicio), hasta que la cantidad coincida con la seleccionada.

Te paso la versión corregida:

JavaScript

```javascript
  const cantidad = parseInt(cantidadColores.value);
  let colores = Array.from(paleta.children);
  const actuales = colores.length;

  // 🔹 Si faltan colores → agregar
  if (actuales < cantidad) {
    for (let i = actuales; i < cantidad; i++) {
      crearColor();
    }
  }

  // 🔹 Si sobran colores → eliminar SOLO los desbloqueados desde el final
  if (actuales > cantidad) {
    // Recorremos de atrás hacia adelante
    for (let i = colores.length - 1; i >= 0; i--) {
```

Uso de la IA - Prompt 3

Prompt
Como debo hacer para cumplir con el soporte de HSL

Resultado de la IA

1 Elegir el formato

Podés agregar un select para que el usuario elija entre **HEX** o **HSL**:

HTML

```html
<label for="formatoColor">Formato de color:</label>
<select id="formatoColor">
  <option value="hex" selected>HEX</option>
  <option value="hsl">HSL</option>
</select>
```

Esto va junto al select de cantidad de colores en tu HTML.

---

2 Modificar la función de generación de colores

Actualmente solo generás HEX. Hay que agregar la opción HSL:

JavaScript

```javascript
const formatoColor = document.getElementById("formatoColor"); // Selección de formato

function generarColorAleatorio() {
  if (formatoColor.value === "hex") {
    // HEX
    const letras = "0123456789ABCDEF";
```

Uso de la IA - Prompt 4

Prompt
Tengo publicando en github mi proyecto pero tengo un error
===error===
el erro sucede cuando copio un color, lo pego en otro lugar y me muestra otro color
======

Resultado de la IA

Ah, entiendo 👍. Ese tipo de error normalmente pasa **por cómo estás generando o mostrando los colores** en tu código. Vamos a analizarlo paso a paso.

1️⃣ Posible causa

Si tu función genera colores aleatorios y los muestra en pantalla, el problema suele ser **el formato de copia**. Por ejemplo:

- Estás mostrando el color en **background** (CSS) y copiás desde allí → el valor que se copia puede no ser el mismo que el que esperás.
- O estás usando `innerText` o `textContent` en el HTML, y el valor incluye espacios, mayúsculas o formato diferente (`#ff0000` vs `#FF0000`).

Ejemplo típico de error:

```javascript
const color = generarColor(); // "#ff0000"
element.textContent = color;  // lo ves como "#ff0000"
```

Si luego copiaslo desde un estilo CSS en lugar del texto, el navegador puede convertirlo a `rgb(255, 0, 0)`.

2️⃣ Solución recomendada

Asegurate de **tener el color en texto puro**, en el formato que querés copiar. Por ejemplo:

```javascript
function copiarColor(elemento) {
  const color = elemento.textContent.trim(); // quitar espacios
  navigator.clipboard.writeText(color)
    .then(() => alert(`Color ${color} copiado`))
    .catch(err => console.error("Error al copiar:", err));
}
```

- `textContent.trim()` → evita espacios que alteren el color al pegarlo.
- Siempre usar `writeText(color)` → garantiza que lo que pegás sea exactamente lo que ves.

3️⃣ Otro detalle

Si tu `generarColor()` devuelve **RGBA con decimales** o **transparencia**, el color puede verse distinto al pegarlo en otro lado.

- Para colores hexadecimales fijos: `#RRGGBB` es más seguro.
- Si querés copiar `rgba(...)`, también funciona, pero el pegado depende de la app destino.

Uso de la IA - Prompt 5

Prompt
Tengo mi proyecto en un repositorio en GiHub y quiero ver el historial de commits

Resultado de la IA

Perfecto, tenés varias formas de ver el historial 👇

✅ Básico (lista de commits)

```bash
git log
```

✅ Más resumido (recomendado)

```bash
git log --oneline
```

👉 Te muestra algo así:

```
a1b2c3d style: mejoras UI
e4f5g6h docs: mover README
```

✅ Con gráfico (muy útil)

```bash
git log --oneline --graph --all
```

👉 Ves las ramas y merges de forma visual

✅ Ver cambios de un commit

```bash
git show <id_del_commit>
```

Ejemplo:

```bash
git show a1b2c3d
```

🌐 Alternativa en GitHub

- Entrá a tu repo
- Hacé clic en "**Commits**"
- Ahí ves todo el historial con interfaz visual

Si querés, después te enseño cómo:

- volver a un commit anterior
- deshacer cambios sin romper nada

(eso te salva bastante 😄)